

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:

Владзиевский Евгений

Группа БР 1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
- Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
- Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
- Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1. Функциональные требования

1.1. Управление пользователями (Users)

- **Регистрация** – создание учётной записи с email, паролем, ФИО, телефоном и ролью (tenant по умолчанию). Email должен быть уникальным.
- **Аутентификация** – вход по email и паролю с выдачей токена (JWT или сессионного).
- **Просмотр профиля** – получение информации о текущем пользователе (без пароля).
- **Редактирование профиля** – изменение ФИО, телефона, пароля.
- **Смена роли** (только для администратора) – изменение роли пользователя (tenant ↔ owner).

1.2. Управление объектами недвижимости (Properties)

- **Создание объекта** – владелец может добавить новый объект с указанием типа, названия, описания, цены за день, города, адреса, геокоординат (опционально).
- **Редактирование объекта** – владелец может изменить любые поля своего объекта, кроме owner_id.

- **Архивация / восстановление** – владелец может перевести объект в статус archived (скрыть из поиска) или обратно в active.
- **Удаление объекта** – владелец может удалить объект (каскадно удаляются фотографии, аренды, сообщения).
- **Просмотр списка объектов** – с фильтрацией по городу, типу, цене, датам доступности (исключая пересекающиеся аренды со статусами confirmed или paid), пагинацией.
- **Просмотр деталей объекта** – полная информация + список фотографий + основная фотография.

1.3. Управление фотографиями (PropertyImages)

- **Загрузка фотографий** – владелец может добавить несколько изображений к своему объекту.
- **Установка главного фото** – владелец может отметить одно из изображений как основное (is_main = true). При этом предыдущее главное фото сбрасывается.
- **Удаление фотографии** – владелец может удалить любое фото своего объекта.

1.4. Бронирование (Rentals)

- **Создание бронирования** – арендатор выбирает объект и даты (start_date, end_date). Система проверяет, что объект свободен в этот период, вычисляет total_price = price_per_day * (end_date - start_date). Статус бронирования – pending.
- **Просмотр своих бронирований** – арендатор видит все свои бронирования, владелец – все бронирования на свои объекты, администратор – все.
- **Подтверждение бронирования** – владелец может изменить статус pending → confirmed. Только после подтверждения можно оплатить.
- **Оплата бронирования** – после подтверждения арендатор может инициировать платеж. При успешной транзакции статус аренды меняется на paid.

- **Отмена бронирования** – арендатор может отменить бронирование, если оно pending или confirmed (до оплаты). Владелец может отменить confirmed (например, форс-мажор). Отмена возможна только до даты начала аренды.
- **Завершение аренды** – по истечении end_date система автоматически (или вручную администратором) переводит статус paid → finished. Либо пользователь может подтвердить завершение.
- **Удаление бронирования** – администратор может удалить запись (например, при нарушении правил).

1.5. Сообщения (Messages)

- **Отправка сообщения** – арендатор или владелец могут написать друг другу по конкретному объекту.
- **Просмотр диалогов** – пользователь видит список чатов (с последним сообщением) по объектам, в которых он участвует.
- **Просмотр истории сообщений** – получение всех сообщений между двумя пользователями по конкретному объекту.

1.6. Транзакции (Transactions)

- **Создание транзакции** – при оплате бронирования создаётся запись с суммой, методом оплаты, статусом success или failed.
- **Просмотр истории платежей** – пользователь может видеть свои транзакции.

1.7. Административные функции

- **Управление типами недвижимости** – CRUD для property_types (доступно только администратору).
- **Просмотр статистики** – количество объектов, бронирований, выручка за период (доступно администратору).

2. Нефункциональные требования

2.1. Производительность и масштабируемость

- **Время ответа API** – не более 200 мс для 95% запросов (при стандартной нагрузке).
- **Поддержка пагинации** для всех списков (limit/offset или cursor).
- **Кэширование** часто запрашиваемых данных (список объектов, типы) через Redis (TTL 5–10 минут).
- **Индексы БД** на внешние ключи, поля фильтрации (city, price_per_day, start_date, end_date).
- **Горизонтальное масштабирование** – бэкенд должен быть stateless

2.2. Безопасность

- **Хэширование паролей** – bcrypt (cost ≥ 10).
- **JWT токены** с временем жизни (access token – 15 минут, refresh token – 7 дней).
- **HTTPS** на всех эндпоинтах.
- **Защита от SQL-инъекций** – использование параметризованных запросов (ORM/query builder).
- **Валидация входных данных** – на уровне схемы + проверки бизнес-логики.
- **CORS** – разрешены только доверенные домены.
- **Rate limiting** – не более 100 запросов в минуту с одного IP (для неавторизованных) и 200 – для авторизованных.
- **Ролевая модель** – middleware проверяет права доступа к ресурсам (владелец может менять только свои объекты).

2.3. Надёжность и отказоустойчивость

- **Резервное копирование БД** – ежедневное с retention 30 дней.

- **Retry и idempotency** для платёжных операций (уникальный ключ идемпотентности).

2.4. Наблюдаемость (Observability)

- **Логирование** всех запросов (method, path, status, duration) и критических операций (оплата, удаление).
- **Метрики** – количество запросов, ошибок, времени ответа (Prometheus + Grafana).
- **Tracing** – correlation ID для каждого запроса.

2.5. Документация

- **OpenAPI (Swagger)** – актуальная документация всех эндпоинтов.
- **Примеры запросов/ответов** в документации.

Вывод

Разработана спецификация **REST API** для сервиса аренды недвижимости, полностью соответствующий схеме БД.

API охватывает все ключевые сущности: пользователи, объекты, фотографии, бронирования, сообщения, транзакции, типы недвижимости.